

# Credit Default — Project Plan

**Group:** Oscar Butler, Alex, Louis · **Language:** Python · **Repo:** [github.com/Oscar339/dst-sprint0](https://github.com/Oscar339/dst-sprint0) · **Due:** Wed 7 Oct 2026, 12:00

## GOALS

A literature review of the resources for doing data science in **consumer credit risk**. Can we predict whether a borrower defaults, and how do the available books, papers and code tackle it?

### Survey the field

Find a broad range of resources with code, run them, and cite them properly.

### Compare approaches

Traditional scorecards against machine learning, on the same data.

### Make it our own

Change the examples to make our own visualisations of both datasets.

### Set up Sprint 1

Choose models and performance measures we will build on for the assessed sprint.

## MATERIALS

| Item                        | Detail                                                                                                                                                                                                                                                                              |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Taiwan credit cards</b>  | 30,000 cardholders, 23 features, label = default next month (22%). UCI dataset 350; Yeh & Lien (2009).                                                                                                                                                                              |
| <b>German Credit</b>        | 1,000 loans, 20 attributes, good/bad (30% bad), with a published 5:1 cost matrix. UCI dataset 144.                                                                                                                                                                                  |
| <b>Where the data lives</b> | data/raw/, exactly as published and never edited; sources and checksums in data/README.md. Not committed, so each person downloads it.                                                                                                                                              |
| <b>Tools</b>                | Python 3.13: pandas, NumPy, Matplotlib, SciPy, scikit-learn, Jupyter; requirements.txt in the repo.                                                                                                                                                                                 |
| <b>Code resources</b>       | <i>optbinning</i> and <i>scorecardpy</i> (scorecards); scikit-learn's ensemble and calibration guides and its cost-sensitive example on German Credit; the <i>hmeasure</i> package.                                                                                                 |
| <b>Literature</b>           | Hand & Henley (1997) and Lessmann et al. (2015), reviews of credit-scoring methods; Thomas, Crook & Edelman, <i>Credit Scoring and Its Applications</i> ; Siddiqi, <i>Intelligent Credit Scoring</i> ; Gneiting & Raftery (2007), proper scoring rules; Hand (2009), the H-measure. |

## WHO DOES WHAT

| Notebook            | Owner        | Content                                                                                           |
|---------------------|--------------|---------------------------------------------------------------------------------------------------|
| 01 Introduction     | Oscar        | The domain and types of credit data; loading and cleaning both datasets; exploratory plots.       |
| 02 Scorecards       | Alex / Louis | The industry-standard approach: binning, weight of evidence, logistic scorecard on German Credit. |
| 03 Machine learning | Alex / Louis | Random forest and gradient boosting on Taiwan; which features matter.                             |
| 04 Evaluation       | Oscar        | Accuracy vs AUC/Gini; Brier score and calibration; cost-weighted loss.                            |
| 05 Wrap-up          | All          | The brief's six questions, drafted in WRAPUP.md first.                                            |

Equity: equal thirds unless agreed otherwise before we start. Each person also keeps their working in their own folder: this is the evidence of contribution.

## DEADLINES

| When                    | What                                                                                         |
|-------------------------|----------------------------------------------------------------------------------------------|
| Sat 26 – Sun 27 Sep     | Agree who owns 02 and 03, and equity; clone the repo, set up Python, download the data.      |
| Mon 28 – Wed 30 Sep     | Find resources, run their examples in your own folder, log them in RESOURCES.md.             |
| Thu 1 Oct               | Check-in after the 10:00 session: what ran, what each notebook will show, split the wrap-up. |
| Fri 2 – Sat 3 Oct       | Write report notebooks with our own plots and citations; draft wrap-up paragraphs.           |
| Sun 4 Oct               | Everything pushed; each person runs the full report from a fresh pull.                       |
| <b>Mon 5 Oct, 12:00</b> | <b>Lock.</b> Final commit with executed notebooks; README reading order and equity complete. |
| Mon 5 – Tue 6 Oct       | Individual reflections, about 500–800 words, written alone.                                  |
| <b>Wed 7 Oct, 12:00</b> | <b>Submit.</b> Each person uploads their reflection and the repo link on Blackboard.         |

## STRATEGY

- **Breadth of literature wins marks.** The unit's example project scored a 2:2 on resources and referencing. Each of us runs at least 4 resources with code, including a paper or book and a repo or tutorial, and cites everything.
- **Run, then change.** Credit comes from our own visualisations and extensions, not from reproducing examples.
- **Same approach, both datasets.** The datasets have different features, so we compare methods and results, not transfer a model between them.
- **Dead ends count.** Record what failed and why; the brief rewards it.
- **One owner per notebook.** Notebooks merge badly. Shared text goes in markdown files with a section per person. Pull before you start, push when you stop.
- **Reproducible.** Paths are relative, the data sources are documented, and nothing is done by hand without a note.
- **Lock early.** Nothing changes after Mon 5 Oct 12:00, so the reflections describe the final report.
- **Build toward Sprint 1** (due Wed 4 Nov): one model each on Taiwan, an agreed test set and performance measure, and a hold-out group of borrowers to test whether results generalise.